

Provincia de Buenos Aires

Dirección General de Cultura y Educación

Dirección Provincial de Educación Secundaria



ESCUELA DE EDUCACIÓN SECUNDARIA TÉCNICA
N° 1

DE VICENTE LÓPEZ, "EDUARDO ADER"

Cerrito 3966, Villa Adelina (1605), Tel. 4735-0174

tecnica1vl@gmail.com

Proyecto:

Malvinas: Voces del frente

Curso y división:

7°2°

Integrantes:

Bruno Fornasar, Alan Bissio y Tomás Onesti

Tema:

Manual de programador

Tabla de versionado

Versión	Fecha	Descripción del cambio	Autor	Estado
v0.1.0	16/06/2026	Se creó el borrador del informe con las instrucciones de cómo instalar el proyecto y trabajar en él.	Bruno Fornasar	Borrador
v1.0.0	23/06/2026	Se entregó el manual con el movimiento de personajes y el ciclo de día.	Bruno Fornasar y Tomás Onesti	Aprobado
v1.1.0	27/07/2026	Se agregó la estructura de directorios.	Bruno Fornasar	Incompleto
v1.2.0	01/09/2026	Se agregó y actualizo la explicación de todas las mecánicas actuales	Tomas Onesti	Revisión

Índice

Tabla de versionado.....	2
Introducción.....	4
Marco tecnológico.....	5
Unity.....	5
Visual Studio Code.....	5
Git.....	5
Organización de directorios.....	6
Instalación.....	6
Configuración de entorno.....	6
Abrir el proyecto en el editor de Unity.....	7
Abrir el proyecto en el editor de código.....	8
Explicación de scripts.....	9
Jugador.....	9
Enemigos.....	16
Día y noche.....	18
Interacciones con el entorno.....	19
Hoguera.....	19

Introducción

En el presente documento se brinda la información necesaria para poder realizar un relevamiento adecuado en el proyecto “*Malvinas - La batalla en el frente*”. Se informará acerca de la infraestructura del proyecto así como de las tecnologías utilizadas para llevarlo a cabo.

Este proyecto es un videojuego 2D desarrollado con el motor gráfico “*Unity*”.

Marco tecnológico

Para poder trabajar adecuadamente en el proyecto necesitamos de las siguientes herramientas:

- Unity versión 2022.3.62f3
- Visual Studio Code de Microsoft o cualquier editor con soporte para Unity (Rider, Visual Studio entre otros)
- Git

Unity

Este es el motor gráfico que se ha utilizado para este proyecto, se ha elegido esta versión ya que la más reciente versión 6 está sujeta a posibles errores por su poco tiempo en el mercado. Su instalación se realiza desde el entorno “*Unity Hub*” donde debería aparecer como la última versión en la lista. En caso de que esto no ocurra puede consultar el sitio oficial de Unity para encontrar una lista con todas las versiones anteriores del motor. Una vez selecciona la indicada se le debería abrir automáticamente el entorno de Unity Hub.

Visual Studio Code

Como se ha indicado anteriormente, la versión que recomendamos es la de Microsoft, ya que esta cuenta con la extensión para desarrollo con Unity, la cual facilitará en gran medida el trabajo. En caso de contar con otra versión del editor que no cuente con esta extensión en la tienda, recomendamos que consulte la documentación de su editor que por lo general incluye una forma de usar extensiones exclusivas de Microsoft.

Git

Para el control de versiones utilizamos Git. Si se desea puede usar el control de versiones de Unity el cual debería dar los mismos resultados.

Organización de directorios

En esta sección se mostrará la estructura de directorios y qué carpetas y archivos son relevantes para el desarrollo.

- /: En este directorio encontrarás los archivos de configuración de Git como el “.gitignore”.
- /Assets/: Este debe ser el directorio más importante de todo el proyecto. Este es en el que se encuentran todos los assets como imágenes y scripts de nuestro juego. Es muy importante ordenar todo en carpetas ya que todos los elementos se encuentran en este directorio.
- /Assets/Scenes/: En esta carpeta se muestran las escenas del videojuego.
- /Assets/interface/: En este directorio se encuentran todos los assets asociados con la interfaz de usuario.
- /Assets/animation/: En esta carpeta se pueden ver todas las animaciones agrupadas por carpetas. Cada una corresponde a una distinta.
- /Assets/Settings/: Esta carpeta guarda la configuración por defecto que crea Unity.
- /Assets/Sprites/: Este directorio es donde guardamos todos los sprites que lleguemos a usar. Lo ideal es agrupar los archivos por carpetas para mantener legible la estructura.
- /Assets/script/: En este directorio colocamos todos los scripts agrupados en carpetas en base a su finalidad. Por ejemplo, aquellos diseñados para controlar al jugador, están todos juntos dentro de la carpeta *“PlayerScripts”*.

Instalación

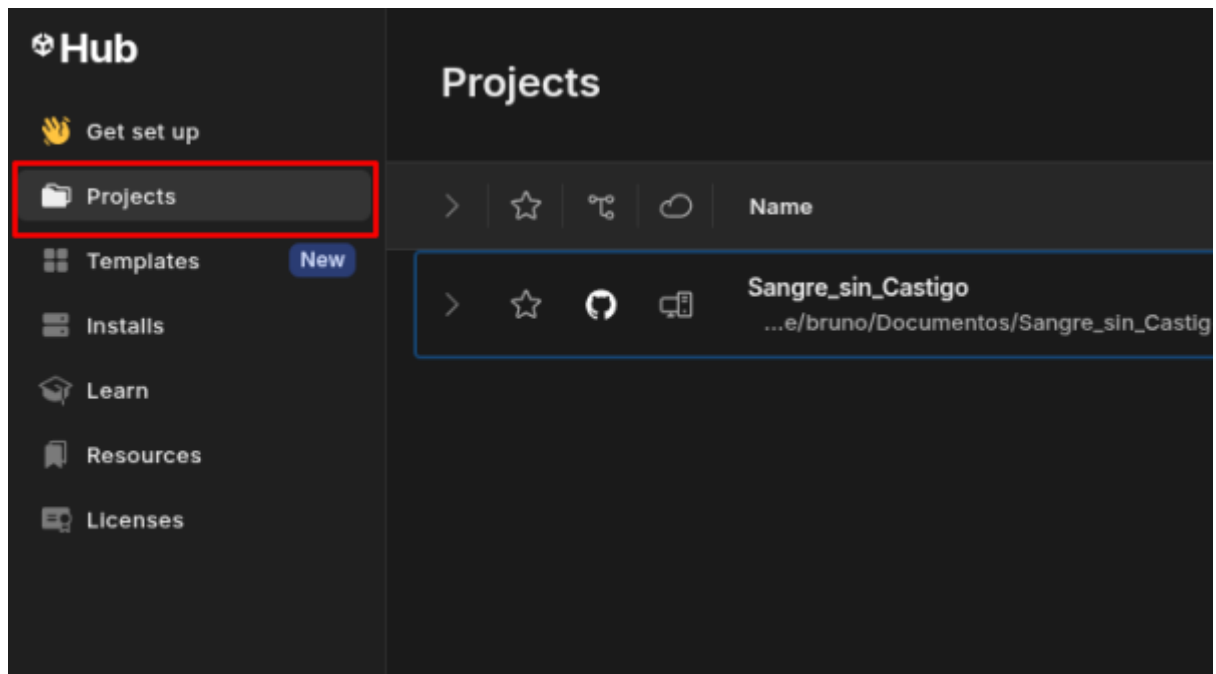
Para instalar el proyecto necesitamos tener una cuenta de Github colaboradora del proyecto para poder acceder al repositorio del mismo. Una vez tengamos acceso, podremos clonar el repositorio con git de la manera que se prefiera.

Configuración de entorno

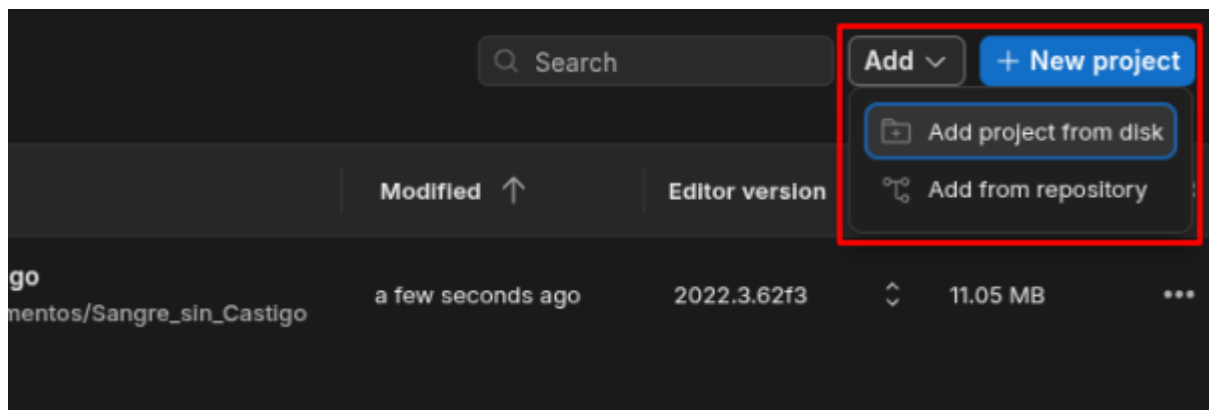
Una vez tenemos el proyecto, ya podemos preparar el entorno para comenzar a modificar el proyecto.

Abrir el proyecto en el editor de Unity

Para abrir el proyecto tenemos que abrir el panel de Unity Hub y dirigirnos a la sección de “Proyectos”.

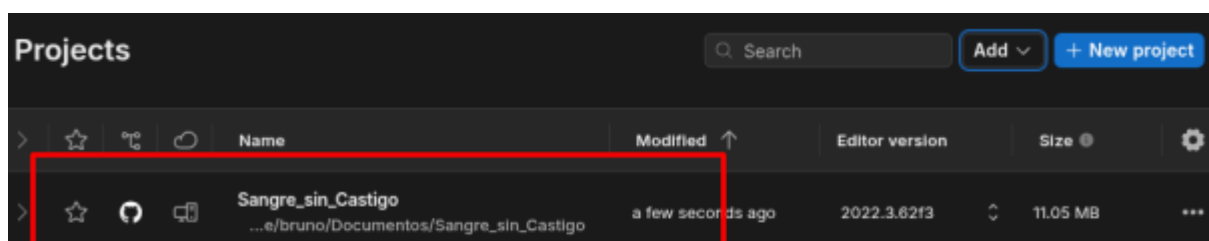


En esta sección abriremos la pestaña “Añadir” y seleccionaremos “Añadir desde el



disco”.

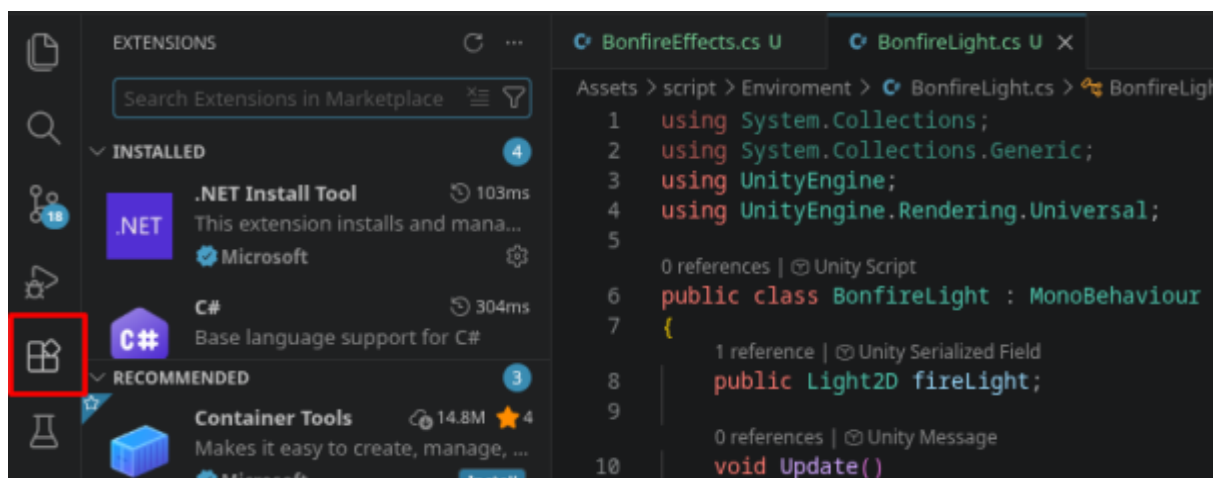
Una vez seleccionemos esta opción se nos abrirá nuestro explorador de archivos por defecto. En el mismo buscaremos la carpeta donde tengamos el proyecto y la seleccionaremos. Tras esto, el proyecto aparecerá en nuestro panel principal.



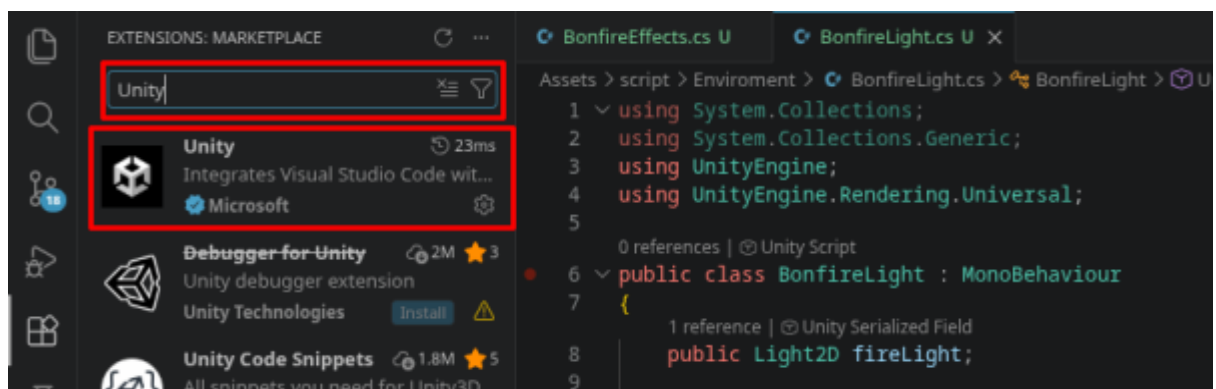
Tras esto ya podemos seleccionarlo para que se nos abra el editor con nuestro proyecto abierto.

Abrir el proyecto en el editor de código

Como Unity no cuenta con editor de código integrado, necesitamos uno externo. Ya especificamos los requisitos que utilizaremos Visual Studio Code. Para trabajar cómodamente con Unity necesitaremos la extensión oficial de Microsoft. Para lo cual deberemos abrir el editor y buscar la sección de extensiones.



En esta sección buscaremos la extensión de Unity y nos aseguraremos de que sea la de Microsoft.



Explicación de scripts

Jugador

Script de movimiento “playermovent”. Script del movimiento del jugador, control de la velocidad del jugador y animaciones de movimiento del jugador.

```
1  using System.Collections;
2  using System.Collections.Generic;
3  using UnityEngine;
4
5  public class topdownmovent : MonoBehaviour
6  {
7      //Solo con esto tenemos el movimiento del personaje
8      //Variables de referencia para la velocidad y direccion
9      [SerializeField] private float speed;
10     [SerializeField] private Vector2 direction;
11     private Rigidbody2D rb2D;
12     //Para la animacion
13     [SerializeField] private float movex;
14     [SerializeField] private float movey;
15     private Animator animator;
16
17     //Funcion para iniciar(siempre en mayuscula, son funciones de unity)
18     private void Start()
19     {
20         animator = GetComponent<Animator>();
21         rb2D = GetComponent<Rigidbody2D>();
22     }
23
24     //Funcion para actualizar(siempre en mayuscula)
25     private void Update()
26     {
27         movex = Input.GetAxisRaw("Horizontal");
28         movey = Input.GetAxisRaw("Vertical");
29         animator.SetFloat("movex", movex); //Animacion en x(izquierda y derecha)
30         animator.SetFloat("movey", movey); //Animacion en y(arriba y abajo)
31         //Parte del funcionamiento de la animacion idle(para estar quieto)
32         if (movex != 0 || movey != 0) {
33             animator.SetFloat("finx", movex);
34             animator.SetFloat("finy", movey);
35         }
36         direction = new Vector2(movex, movey).normalized;
37         // Debug.Log("movex: " + movex.ToString());
38         // Debug.Log("movey: " + movey.ToString());
39     }
40
41     //Otra funcion para actualizar(siempre en mayuscula)
42     private void FixedUpdate()
43     {
44         rb2D.MovePosition(rb2D.position + direction * speed * Time.fixedDeltaTime);
45     }
46 }
```

Script de disparo del jugador. Controla el cómo y con que se dispara(velocidad, cantidad, delay de las balas).

```
1 using System.Collections;
2 using System.Collections.Generic;
3 using UnityEngine;
4
5 public class controlCamara : MonoBehaviour
6 {
7     [SerializeField] private GameObject bulletP; //Prefab de la bala
8     [SerializeField] private float bulletsSpeed; //Velocidad
9     [SerializeField] private float shootDelay; // un pequeño delay
10
11     private float lastShoot; //Detecta el ultimo disparo para poner el delay
12
13     void Start()
14     {
15     }
16
17     void Update()
18     {
19         float horShoot = Input.GetAxisRaw("ShootHorizontal"); //Teclas de disparo horizontal y abajo la vertical(alfinal lo hice con flechas, si quieren lo puedo cambiar)
20         float verShoot = Input.GetAxisRaw("ShootVertical");
21
22         if((horShoot != 0 || verShoot != 0) && Time.time > lastShoot + shootDelay) //Establece cuando se ejecuta el disparo teniendo en cuenta el delay
23         {
24             shoot(horShoot, verShoot); //Dispara
25             lastShoot = Time.time; //Guarda el ultimo disparo(para el delay de arriba(esto tambien por si hacemos varias armas lo vamos modificando y asi))
26         }
27     }
28
29     void shoot(float x, float y)
30     {
31         GameObject bullet = Instantiate(bulletP, transform.position, transform.rotation) as GameObject; //instanciamos las balas
32         bullet.AddComponent<Rigidbody2D>().gravityScale = 0; //sin gravedad por ser un top-down
33         bullet.GetComponent<Rigidbody2D>().velocity = new Vector2(x, y) * bulletsSpeed; //la velocidad de las balas y direccion al disparar
34     }
35 }
```

Control de duración de las balas

```
Code Blame 27 lines (20 loc) · 428 Bytes
1 using System.Collections;
2 using System.Collections.Generic;
3 using UnityEngine;
4
5 public class bulletController : MonoBehaviour
6 {
7     public float lifetime;
8
9     void Start()
10     {
11         StartCoroutine(DeathDelay());
12     }
13
14     // Update is called once per frame
15     void Update()
16     {
17     }
18
19
20     IEnumerator DeathDelay()
21     {
22         yield return new WaitForSeconds(lifetime);
23         Destroy(gameObject);
24     }
25
26 }
```

Funcionamiento de las barras visuales de vida y frío.

```
1  using System.Collections;
2  using System.Collections.Generic;
3  using UnityEngine;
4  using UnityEngine.UI;
5
6  ✓ public class HpBar : MonoBehaviour
7  {
8
9      public Image fillBar;
10     public PlayerStates playerStates;
11     public float maxHP;
12     // Start is called before the first frame update
13     void Start()
14     {
15         maxHP = 100;
16     }
17
18     // Update is called once per frame
19     void Update()
20     {
21         fillBar.fillAmount = playerStates.HP / maxHP;
22     }
23 }
```

```
1  using System.Collections;
2  using System.Collections.Generic;
3  using UnityEngine;
4  using UnityEngine.UI;
5
6  ✓ public class ColdBar : MonoBehaviour
7  {
8      public Image fillBar;
9      public PlayerStates playerStates;
10     public float maxCold;
11     // Start is called before the first frame update
12     void Start()
13     {
14         maxCold = 100;
15     }
16
17     // Update is called once per frame
18  ✓ void Update()
19     {
20         fillBar.fillAmount = playerStates.cold / maxCold;
21         //Debug.Log(playerStates.cold / maxCold);
22     }
23 }
```

Funcionamiento de inventario.

```
6  ✓ public class inventoryScript : MonoBehaviour
7  {
8      [SerializeField] private List<GameObject> Backpack = new List<GameObject>();
9      [SerializeField] private GameObject inventory;
10     public topdownmovent player;
11     public bool lifeInv;
12
13  ✓ void OnTriggerEnter2D(Collider2D collider)
14  {
15      if (collider.CompareTag("item"))
16      {
17          for (int i = 0; i < Backpack.Count; i++)
18          {
19              if (Backpack[i].GetComponent<Image>().enabled == false)
20              {
21                  Backpack[i].GetComponent<Image>().enabled = true;
22                  Backpack[i].GetComponent<Image>().sprite = collider.GetComponent<SpriteRenderer>().sprite;
23                  break;
24              }
25          }
26      }
27  }
28
29  ✓ void Start()
30  {
31      player = FindObjectOfType<topdownmovent>();
32      inventory.SetActive(false);
33      lifeInv = false;
34  }
35
36  ✓ void Update()
37  {
38      if (Input.GetKeyDown(KeyCode.I))
39      {
40          lifeInv = !lifeInv;
41          inventory.SetActive(lifeInv);
42      }
43
44      if(lifeInv == true)
45      {
46          player.speed = 0;
47      }
48      else
49      {
50          player.speed = 6;
51      }
52  }
53 }
```

Script de GameManager que se usa para controlar el cambio de escenas al morir y mostrar la pantalla de muerte. Primera parte con: start, variables, awake y librerías.

```
1  using System.Collections;
2  using System.Collections.Generic;
3  using UnityEngine;
4  using UnityEngine.SceneManagement;
5  using TMPPro;
6  using UnityEngine.UI;
7
8  public class GameManager : MonoBehaviour
9  {
10     public static GameManager instance;
11     public GameObject gameOver;
12     public TextMeshProUGUI gameOverText;
13     public Button reset;
14     public Button menu;
15
16     private bool gameOverActive;
17
18     void Awake()
19     {
20         if (instance == null)
21         {
22             instance = this;
23         }
24         else
25         {
26             Destroy(gameObject);
27         }
28     }
29
30
31
32     // Start is called before the first frame update
33     void Start()
34     {
35         if (gameOver != null)
36         {
37             gameOver.SetActive(false);
38         }
39         if (reset != null)
40         {
41             reset.onClick.AddListener(ResetScene);
42         }
43         if (menu != null)
44         {
45             menu.onClick.AddListener(Menu);
46         }
47     }
```

Script de Game Manager. Segunda parte del sistema de muerte con: Update, función de reinicio, función de salir al menú y función de activación de la pantalla de muerte.

```
50  void Update()
51  {
52      if (gameOverActive)
53      {
54          if (Input.GetKeyDown(KeyCode.R))
55          {
56              ResetScene();
57          }
58          if (Input.GetKeyDown(KeyCode.Escape))
59          {
60              Menu();
61          }
62      }
63  }
64
65  public void GameOver()
66  {
67      if (gameOverActive) return;
68      gameOverActive = true;
69
70      if (gameOver != null)
71      {
72          gameOver.SetActive(true);
73      }
74
75      if (gameOverText != null)
76      {
77          gameOverText.text = "You Dead\n\nR - Reiniciar\nEsc - Volver al Menu";
78      }
79
80  }
81
82  public void ResetScene()
83  {
84      Time.timeScale = 1f;
85      SceneManager.LoadScene(SceneManager.GetActiveScene().name);
86  }
87
88  public void Menu()
89  {
90      Time.timeScale = 1f;
91      SceneManager.LoadScene("MainMenu");
92  }
93  }
```

En PlayerStates se llama a la función de activación de la pantalla de muerte al morir con la función dead.

```
public void dead()
{
    if (HP <= 0)
    {
        Debug.Log("Funciono 1");
        if(GameManager.instance != null)
        {
            Debug.Log("Funciono 2");
            GameManager.instance.GameOver();
        }
    }
}
```

Enemigos

Movimiento de los enemigos(seguimiento).

```
1  using System.Collections;
2  using System.Collections.Generic;
3  using UnityEngine;
4
5  public class enemigo : MonoBehaviour
6  {
7      public Transform player;//Jugador
8      [SerializeField] private float distance;//Distancia minima
9      [SerializeField] private float enemySpeed;//Velocidad(hay que poner valores altos(supongo por el deltaTime))
10     private Rigidbody2D rb;//Enemigo
11     private Vector2 movement;//Movimiento
12
13     void Update()
14     {
15         Follow();
16     }
17
18     void Follow()//Funcion para seguir al jugador
19     {
20         if (Vector2.Distance(transform.position, player.position) > distance)//Medimos la distancia entre el jugador y el enemigo
21         {
22             transform.position = Vector2.MoveTowards(transform.position, player.position, enemySpeed * Time.deltaTime);
23         }
24     }
25 }
```

Disparo del enemigo. Detección al jugador, disparó con velocidad y delay variable.

```
1  using System.Collections;
2  using System.Collections.Generic;
3  using UnityEngine;
4
5  public class enemyBullet : MonoBehaviour
6  {
7      //Variables de la bala enemiga
8      [SerializeField] private float speedBullet;
9      private Transform player;
10     private Rigidbody2D rb;
11
12     void Start()
13     {
14         player = GameObject.Find("player").transform;//Traemos el Transform del jugador
15         rb = GetComponent<Rigidbody2D>();
16         EnemyShoot();
17     }
18
19     void EnemyShoot()//Funcion de disparo
20     {
21         Vector2 shoot = (player.position - transform.position).normalized;//Le indicamos a donde se mueve la bala
22         rb.velocity = shoot * speedBullet;//Le damos una velocidad a la bala
23         StartCoroutine(DestroyBullet());
24     }
25
26     IEnumerator DestroyBullet()//Corrutina para destruir las balas
27     {
28         float destroyTime = 1f;
29         yield return new WaitForSeconds(destroyTime);//Espera y ejecuta la destruccion
30         Destroy(gameObject);//Destruye la bala
31     }
32
33     private void OnCollisionEnter2D()
34     {
35         Destroy(gameObject);
36     }
37 }
```

Control de disparo del enemigo.

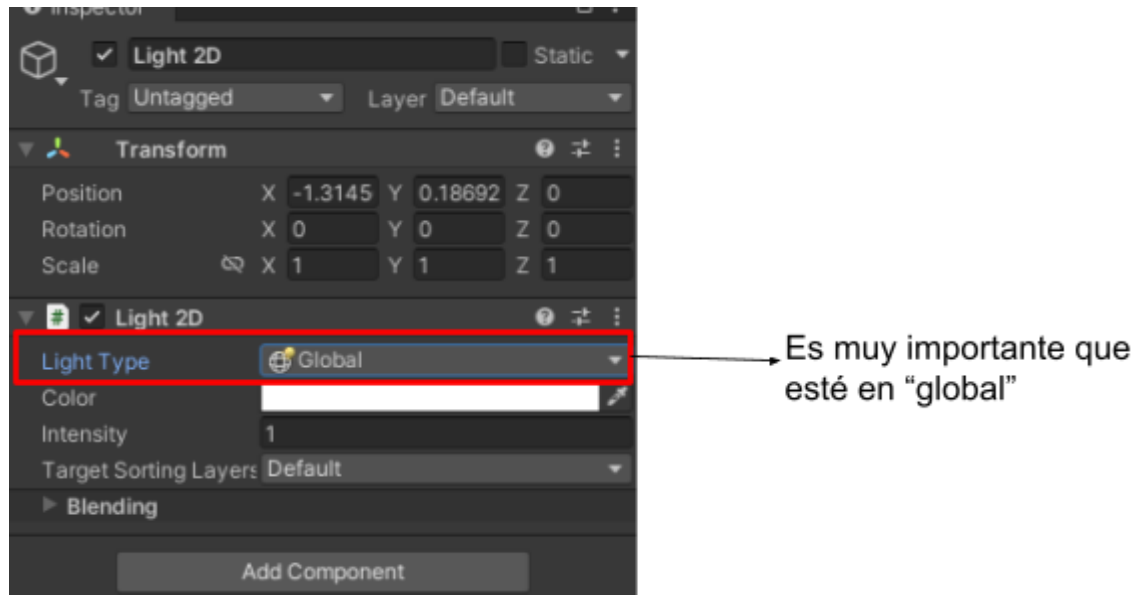
```
1 using System.Collections;
2 using System.Collections.Generic;
3 using UnityEngine;
4
5 public class controllerEnemyShoot : MonoBehaviour
6 {
7     //Variables para controlar el disparo
8     [SerializeField] private GameObject prefab;
9     [SerializeField] private float delay;
10    [SerializeField] private Transform player;
11    [SerializeField] private float range;
12
13    void Start()
14    {
15        StartCoroutine(Shoot());
16    }
17
18    IEnumerator Shoot() //Corrutina de disparo
19    {
20        while (true) //Siempre se debe ejecutar
21        {
22            yield return new WaitForSeconds(delay); //Espera el delay y ejecuta el disparo
23            if (Vector2.Distance(transform.position, player.position) <= range) //Se ejecuta el disparo solo si se esta en el rango
24            {
25                Instantiate(prefab, transform.position, Quaternion.identity); //Instancia las balas para que vayan al enemigo
26            }
27        }
28    }
29 }
```

Movimiento patrullaje.

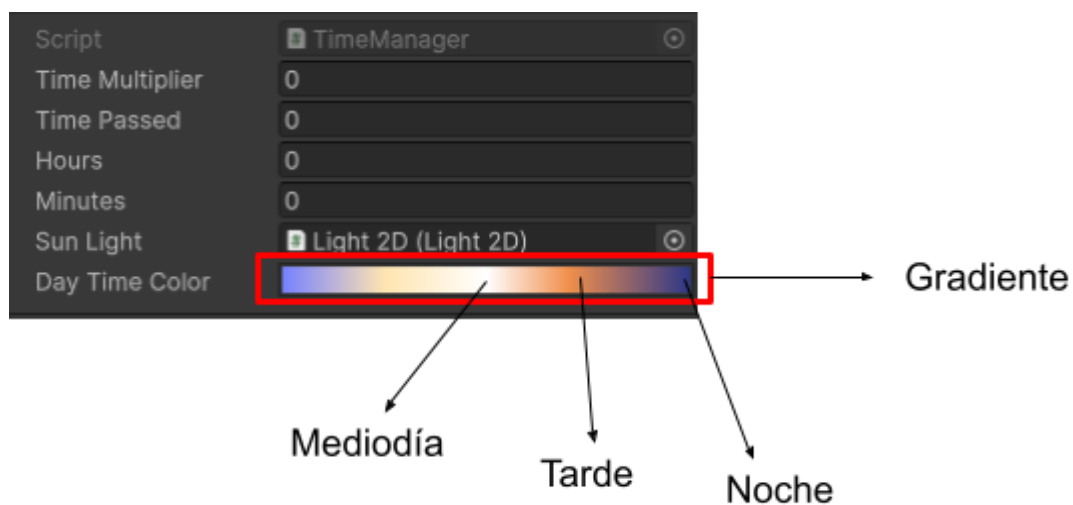
```
1 using System.Collections;
2 using System.Collections.Generic;
3 using UnityEngine;
4
5 public class script_enemigo2 : MonoBehaviour
6 {
7     [SerializeField] private float speed;
8     [SerializeField] private float waitTime;
9     private bool waiting;
10    [SerializeField] private Transform[] waypoints;
11    private int waypointActual;
12    private float distanciaMinima = 0.1f;
13
14    void Update()
15    {
16        float distancia = Vector2.Distance(transform.position, waypoints[waypointActual].position);
17        if (distancia > distanciaMinima)
18        {
19            transform.position = Vector2.MoveTowards(transform.position, waypoints[waypointActual].position, speed * Time.deltaTime);
20        }
21        else if (!waiting)
22        {
23            StartCoroutine(Wait());
24        }
25    }
26
27    IEnumerator Wait()
28    {
29        waiting = true;
30        yield return new WaitForSeconds(waitTime);
31
32        waypointActual++;
33        if (waypointActual == waypoints.Length)
34        {
35            waypointActual = 0;
36        }
37
38        waiting = false;
39    }
40 }
41 }
```

Día y noche

En la jerarquía podemos encontrar el objeto “Light 2D” el cual es el que aplica una especie de filtro en todo el mapa para que se vea como si fuera de día o de noche.



Este objeto luego se lo asignamos mediante el editor de Unity al script de “[TimeManager.cs](#)”. En este se podrá ver una barra con distintos colores, estos representan la iluminación que habrá en cada momento del día. Para entender el funcionamiento hay que tener en cuenta que el primer punto de esa gradiente es 0 y el último es 1, entonces hay que dividir la hora actual (del videojuego) por 24. De esta forma obtenemos un valor que encaja perfectamente en este gradiente.



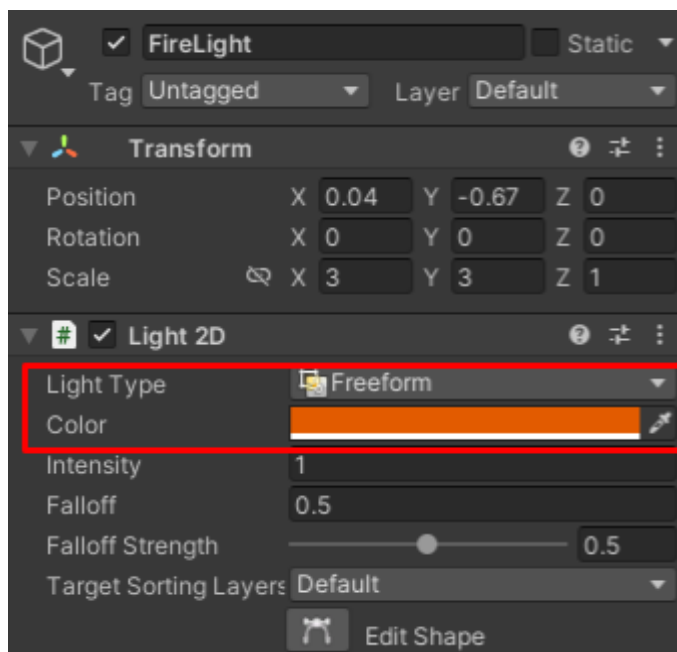
Interacciones con el entorno

En esta sección se detallarán algunos detalles acerca de las posibles interacciones que pueden existir con el entorno con el fin de comprender cuál es la lógica detrás.

Hoguera

La hoguera es un objeto que cuenta con otros dos contenedores clave, el primero es simplemente uno que conlleva el sprite del elemento y el segundo es el que lleva el cuerpo que emite luz. Es importante destacar que este elemento cuenta con dos áreas de colisión. La primera es una rectangular que se usa para que el jugador no traspase el objeto, mientras que la segunda es un circular marcada como un trigger, la cual se usa para saber cuándo el usuario pisa el área de interacción.

Es muy importante definir el tipo de luz en “Freeform” para que solo abarque a la



zona del objeto y no a todo el mapa. Luego se puede cambiar el color de la luz.